Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЕТ
к лабораторной работе №2
на тему:
«ТАЙМЕР»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Функциональные требования:

1 Пользователь может создавать различные таймерные последовательности и переключаться между ними (используйте GSON или SQLite для хранения данных).

2 Главная страница – список последовательностей. CRUD-операции для них (создание, чтение, обновление, удаление).

3 Свойства последовательности: название, цвет.

4 Страница таймера: оставшееся время текущей фазы, список предстоящих фаз, элементы управления для паузы, перехода вперед/назад по последовательности или выхода со страницы и отмены таймера.

5 Страница редактирования: изменение продолжительности каждой фазы (работа, отдых, разминка, заминка), изменение количества повторений фаз и периодов отдыха между ними, определение свойств последовательности.

6 Во время работы переключение каждой фазы должно сопровождаться звуковым сигналом для оповещения пользователя.

7 Таймер не должен останавливаться, когда пользователь покидает приложение (см. Services).

8 Страница настроек (реализованная с помощью библиотеки AndroidX Preference): переключение темы (дневная / ночная), изменение размера шрифта в приложении (без перезагрузки приложения), кнопка для очистки всех сохраненных данных, переключение локали приложения (выбор как минимум между двумя языками).

9 Приложение должно иметь экран-заставку (splash screen) с названием приложения или графикой, чтобы занять пользователя во время загрузки приложения.

В отчете по лабораторной работе представьте следующие диаграммы: Диаграмма вариантов использования (Use-Case diagram), Диаграмма активности (Activity diagram).

2 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе реализации мобильного приложения была использована современная многослойная архитектура, сочетающая принципы Clean Architecture с паттернами MVVM и MVI для организации реактивного пользовательского интерфейса. Основной стек технологий включает язык Kotlin, фреймворк Jetpack Compose и библиотеку Koin для внедрения зависимостей.

Применение концепции однонаправленного потока данных позволило централизовать состояние экранов в ViewModel, где каждое действие пользователя обрабатывается как отдельное намерение (Intent). Согласно разработанной диаграмме активности, представленой на рисунке 2.1 отправной точкой взаимодействия является экран списка последовательностей.

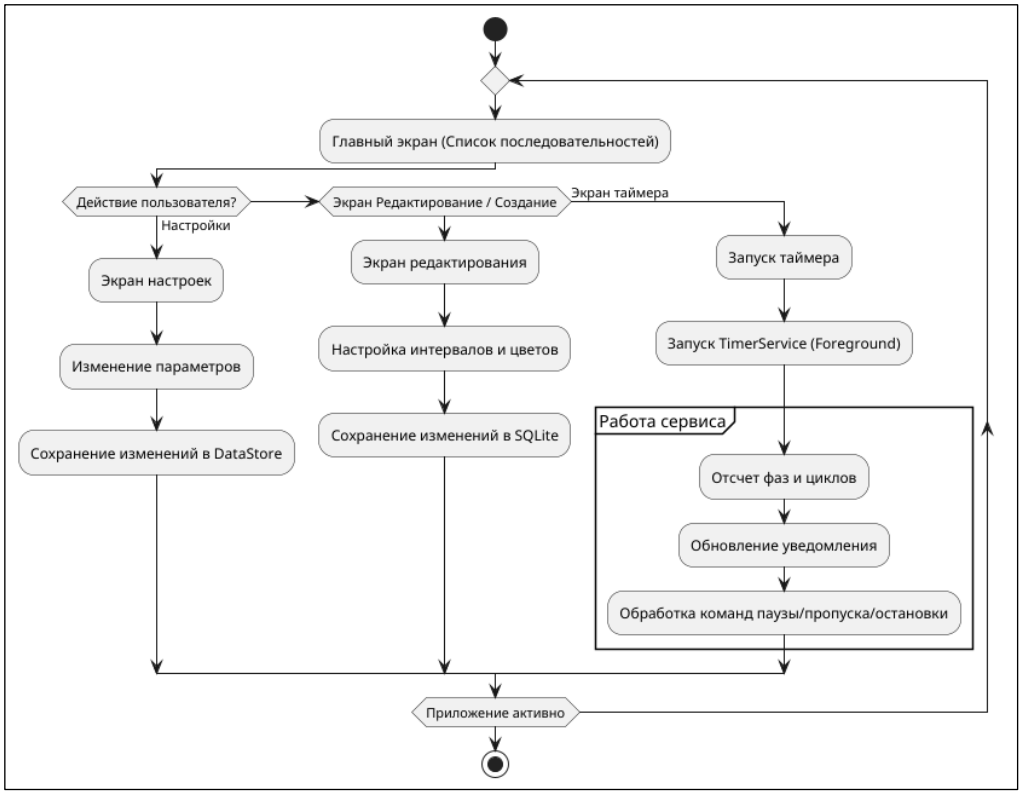

Рисунок 2.1 – Диаграмма активности приложения

Рисунок 2.2, где отображаются сохраненные таймеры. Как раз таки этот экран и является главным в приложении. На диаграмме это главный экран или список последовательностей. В дальнейшем будем называть его список таймеров.

Рисунок 2.2 – Список таймеров

При переходе к созданию/редактированию элемента открывается экран, в котором логика настройки фаз изолирована в EditSequenceViewModel и соответствующих UseCase-классах. Этот экран представлен на рисунке 2.3.

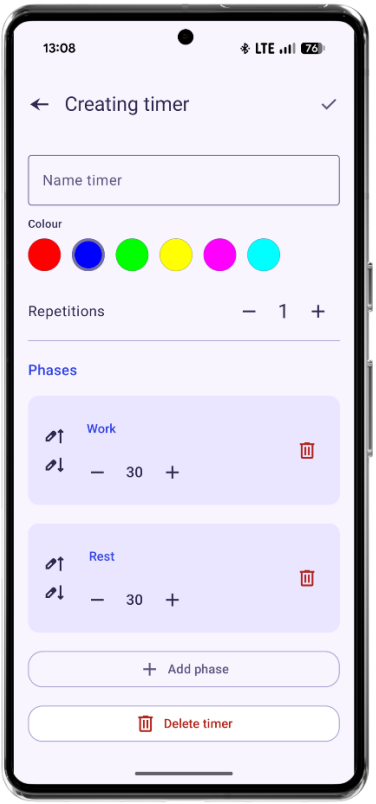

Рисунок 2.3 – Создание таймера

Для настроек, экран которых представлен на рисунке 2.4, используется SettingsRepository, обеспечивающий смену темы, языка и удаление всех таймерых последовательностей приложения.

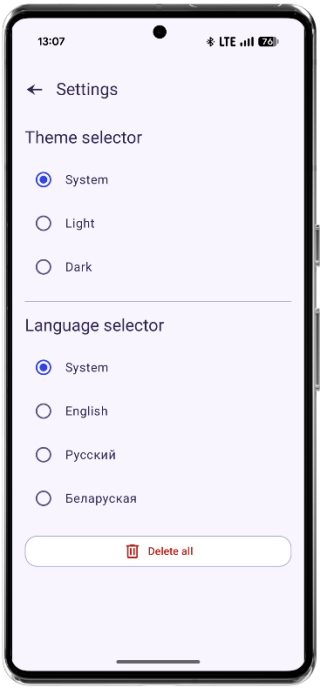

Рисунок 2.4 – Создание таймера

Процесс тренировки визуализируется на экране таймера, что можно увидеть на рисунке 2.5, работа которого неразрывно связана с TimerService.

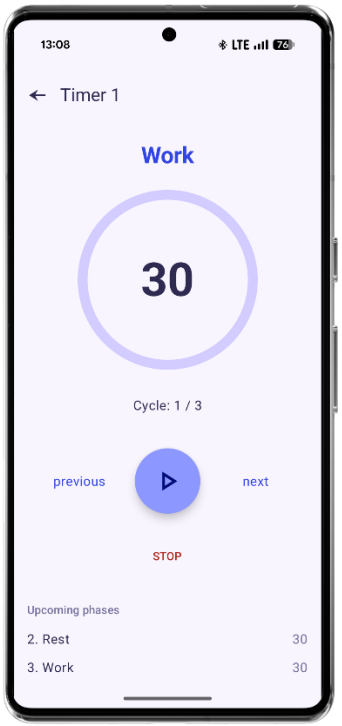

Рисунок 2.5 – Экран таймера

Как отражено в блоке работы сервиса на диаграмме, он функционирует как Foreground Service. Это гарантирует точность отсчета в фоне и позволяет управлять процессом через интерактивное уведомление, как на рисунке 2.6, при этом состояние таймера синхронизируется между сервисом и UI в реальном времени.

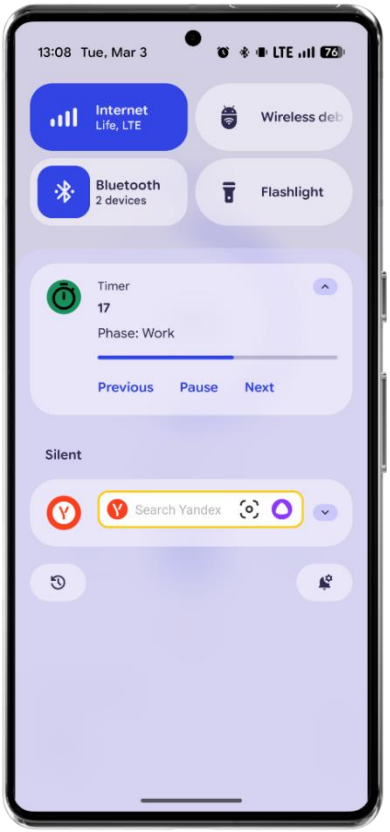

Рисунок 2.6 – Foreground Service

Usecase-диаграмма связывает интерфейс с бизнес-логикой, показывая логическую полноту вашего приложения. Увидеть её можно на рисунке 2.7.

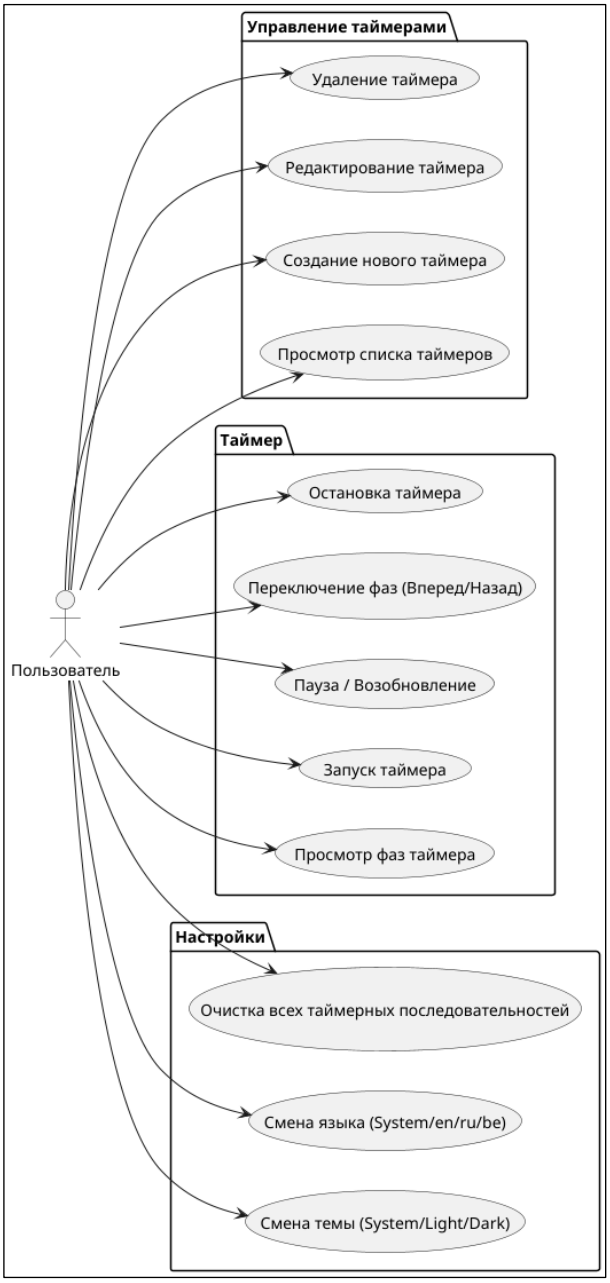

Рисунок 2.7 – Usecase-диаграмма

ВЫВОД

Разработанное приложение демонстрирует эффективное применение архитектурного синтеза MVVM и MVI, что обеспечило строгий контроль над состоянием интерфейса и упростило обработку асинхронных событий от таймера. Использование диаграммы активности позволило спроектировать циклическую навигацию, исключающую потерю данных при переходах между экранами настроек и редактирования. Реализация ключевой логики через Foreground Service обеспечила бесперебойную работу приложения в условиях ограничений Android на фоновые процессы, что подтверждается корректным поведением системы при взаимодействии с уведомлением. Применение Clean Architecture позволило изолировать бизнес-логику от фреймворка, сделав код пригодным для дальнейшего масштабирования и тестирования. Итоговый программный продукт полностью реализует заявленный функционал интервального тренажера и соответствует стандартам современной мобильной разработки.